

P1467R0

Extended floating-point types

Published Proposal, 2019-01-21

This version:

<https://wg21.link/P1467R0>

Authors:

[Michał Dominiak](#) (NVIDIA)

[David Olsen](#) (NVIDIA)

Audience:

SG6, EWG, LEWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1	Abstract
2	Motivation
3	Proposed approach
3.1	Finer design details
3.1.1	Floating-point conversion rank
3.1.2	Narrowing conversions
3.1.3	Support throughout the library
4	Proposed wording
4.1	Core language
4.2	Library

References

Informative References

§ 1. Abstract

This proposal is the less evolutionary part of [\[P1468\]](#), that attempts to ultimately provide the same functionality of [\[P0192\]](#) in a way that we expect to be more acceptable to the committee than the previous attempt.

This paper introduces the notion of *extended floating-point types*, modeled after extended integer types. To accomodate them, this paper also attempts to rewrite the current rules for floating-point types, to enable well-defined interactions between all the floating-point types. The end goal of this paper, together with [\[P1468\]](#), is to have a language to enable `<cstdlibint>`-like aliases for implementation specific floating point types, that can model more binary layouts than just a single fundamental type (the previously proposed `short float`) can provide for.

It also attempts to rewrite existing specification for both the core language and the library to not spell out all standard floating-point types every time.

§ 2. Motivation

The motivation for the general effort of this paper is the same as for [\[P0192\]](#), so we decided to avoid repeating it here, for brevity.

The motivation for taking the currently proposed approach comes from the result of discussion on the previous paper. Several people raised concerns about introducing just a single new fundamental type with not well defined layout; those same people were not satisfied with the option of having a dual ABI for that type, when for instance both IEEE-754 `binary16` and `bfloat` are needed in the same application.

This paper legitimizes implementation-specific floating-point types, which makes standardizing an existing practice an additional motivation for solving the need in the way described below.

§ 3. Proposed approach

In a nutshell:

1. Introduce the notion of *extended floating-point types*.
2. Redefine usual arithmetic conversions in terms of *floating-point conversion rank*, closely modeled after the integer equivalent.
3. Redefine narrowing conversions for floating-point types, to be defined in terms of value ranges, instead of being fixed for the standard floating-point types.
4. Rewrite as much of the standard library spec to use the new notion where it is possible and makes sense.

§ 3.1. Finer design details

Here's a list of the details of the design of this paper that we think are important; we'd like guidance on whether the committee likes the decision we've made, or if a change to them is requested; please consider them as proposed polls to determine that.

§ 3.1.1. Floating-point conversion rank

At this time, the paper uses the *range of finite values* of a given floating-point type for determining the conversion rank; this is motivated by the fact that converting a value to a type that can't represent it is undefined behavior. It is implementation-defined if a floating-point type can represent infinities or not; if they can, then the UB goes away, but we think that this is the useful way to determine the rank, even when the range of values is the entire set of real numbers, therefore the use of the notion of range of **finite** values. There is probably more acceptable behaviors, but this seems to be the most acceptable of them for the authors.

Since the definition this paper gives orders types by the relation of the ranges of finite values of different types, we included an item for when two types have ranges of finite values that are neither a subset nor a superset of each other. This doesn't seem necessary in reality, but we decided to include it for completeness of the rules.

§ 3.1.2. Narrowing conversions

This paper proposes to change the rules of narrowing conversions in a way that may introduce changes to what expressions are well- or ill-formed on systems, where `float` and `double`, and/or `double` and `long double`, have the same size and layout.

Currently, the rule for narrowing conversion reads: `long double` to `double` or `float` and `double` to `float` is a narrowed conversion. After the proposed change, that will only be the case if those types have different ranges of finite values. This change is made to simplify the rules; the rule that determines if a conversion is narrowing or not based on the range of finite values is necessary for extended floating-point types, so it needs to appear in the text, so we decided to change the old rule and unify it with the new one; the situation where they give a different result seems strange enough to justify this decision.

There's another possible approach: to define that a floating-point conversion from a type with a higher floating-point conversion rank to a type with a lower floating-point conversion rank is always narrowing. This mostly follows the rule above, however it preserves the current narrowing conversion relations between standard floating-point types. This is not the approach currently worded by this paper, but we have no objections to move to this approach if it is preferred by the committee.

§ 3.1.3. Support throughout the library

Extended floating-point types are supported in some part of the library, that is: `<cmath>` (because having access to operations on shorter `float`s is the entire point of this feature), `<complex>` (for the same reason), and `<charconv>` (because some way of I/O should be available for them, and because the existing spec supports extended integer types already). They are not supported in `num_get` and `num_put`, because (a) properly supporting them would require an ABI break (and then again every time the implementation adds an extended floating-point type) and (b) because extended integer types are not supported there. Similarly, no stream support is included in this paper.

§ 4. Proposed wording

The wording changes in this paper are relative to N4791.

§ 4.1. Core language

Modify Fundamental types [**basic.fundamental**] paragraph 12:

There are three *standard* floating-point types: `float`, `double`, and `long double`. The type `double` provides at least as much precision as `float`, and the type `long double` provides at least as much precision as `double`. The set of values of the type `float` is a subset of the set of values of the type `double`; the set of values of the type `double` is a subset of the set of values of the type `long double`. The value representation of *standard* floating-point types is implementation-defined. *There may also be implementation-defined extended floating-point types. The range from the lowest finite value representable by a floating-point type to the maximum finite value representable by that type is called the range of finite values of that type. The standard and extended floating-point types are collectively called floating-point types.* [...]

Rename `Integer conversion rank` [`conv.rank`] to `Conversion ranks` and insert a new paragraph at the end:

2. *Every floating-point type has an floating-point conversion rank defined as follows:*

- (2.1) *The rank of a floating point type `T` shall be greater than the rank of any floating-point type whose range of finite values is a subset of the range of finite values of `T`.*
- (2.2) *The rank of `long double` shall be greater than the rank of `double`, which shall be greater than the rank of `float`.*
- (2.3) *The rank of any standard floating-point type shall be greater than the rank of any extended floating-point type with the same range of finite values.*
- (2.4) *The rank of any extended floating-point type relative to another extended floating-point type with the same range of values is implementation-defined, but still subject to the other rules for determining the floating-point conversion rank.*
- (2.5) *For extended floating-point types `T1` and `T2`, if the range of finite values of `T1` is neither a subset nor a superset of the range of finite values of `T2`, the rank of `T1` relative to `T2` is implementation-defined.*
- (2.6) *For all floating-point types `T1`, `T2` and `T3`, if `T1` has greater rank than `T2` and `T2` has greater rank than `T3`, then `T1` shall have greater rank than `T3`.*

[*Note: The floating-point conversion rank is used in the definition of the usual arithmetic conversions ([`expr.arith.conv`]). -- end note*]

Modify Floating-point promotion [`conv.fpprom`] paragraph 1:

1. A prvalue of a floating-point type `float` whose floating-point conversion rank (`[conv.rank]`) is less than the rank of `double` can be converted to a prvalue of type `double`. The value is unchanged.

Modify Usual arithmetic conversions [`expr.arith.conv`] paragraph 1:

- (1.1) If either operand is of scoped enumeration type, no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.
- ~~(1.2) If either operand is of type long double, the other shall be converted to long double.~~
- ~~(1.3) Otherwise, if either operand is double, the other shall be converted to double.~~
- ~~(1.4) Otherwise, if either operand is float, the other shall be converted to float.~~
- (1.2) Otherwise, if either operand has a floating-point type, the following rules shall be applied:
 - (1.2.1) If both operands have the same type, no further conversion is needed.
 - (1.2.2) Otherwise, if one of the operands has a type that is not a floating-point type, that operand shall be converted to the type of the operand with floating-point type.
 - (1.2.3) Otherwise, the operand with the type of lesser floating-point conversion rank shall be converted to the type of the operand with greater rank.
- (1.5) Otherwise, the integral promotions [...]

Modify the definition of narrowing conversions in List-initialization [`dcl.init.list`] paragraph 7 item 2:

- (7.2) ~~from long double to double or float, or from double to float~~ from a floating-point type `T` to another floating-point type whose range of finite values is not a superset of the range of finite values of `T`, except where the source is a constant expression and the actual value after conversion is within the range of finite values that can be represented (even if it cannot be represented exactly), or

§ 4.2. Library

Modify Header `<charconv>` synopsis [`charconv.syn`]:

[...]

```
to_chars_result to_chars(char* first, char* last, *see below* value, i
```

```
to_chars_result to_chars(char* first, char* last, float value);  
to_chars_result to_chars(char* first, char* last, double value);  
to_chars_result to_chars(char* first, char* last, long double value);  
  
to_chars_result to_chars(char* first, char* last, float value, chars_f  
to_chars_result to_chars(char* first, char* last, double value, chars_  
to_chars_result to_chars(char* first, char* last, long double value, c  
  
to_chars_result to_chars(char* first, char* last, float value,  
chars_format fmt, int precision);  
to_chars_result to_chars(char* first, char* last, double value,  
chars_format fmt, int precision);  
to_chars_result to_chars(char* first, char* last, long double value,  
chars_format fmt, int precision);
```

```
to_chars_result to_chars(char* first, char* last, *see below* value);  
to_chars_result to_chars(char* first, char* last, *see below* value, c  
to_chars_result to_chars(char* first, char* last, *see below* value,  
chars_format fmt, int precision);
```

[...]

```
from_chars_result from_chars(const char* first, const char* last,  
                             see below& value, int base = 10);
```

```
from_chars_result from_chars(const char* first, const char* last, floa  
chars_format fmt = chars_format::general)  
from_chars_result from_chars(const char* first, const char* last, doub  
chars_format fmt = chars_format::general)  
from_chars_result from_chars(const char* first, const char* last, long  
chars_format fmt = chars_format::general)
```

```
from_chars_result from_chars(const char* first, const char* last, *see  
chars_format fmt = chars_format::general).
```

[...]

Modify Primitive numeric output conversion [**charconv.to.chars**]:

[...]

```
to_chars_result to_chars(char* first, char* last, float value);  
to_chars_result to_chars(char* first, char* last, double value);  
to_chars_result to_chars(char* first, char* last, long double value);
```

```
to_chars_result to_chars(char* first, char* last, *see below* value);
```

8. *Effects*: `value` is converted to a string in the style of `printf` in the "C" locale. The conversion specifier is `f` or `e`, chosen according to the requirement for a shortest representation (see above); a tie is resolved in favor of `f`.
9. *Throws*: Nothing.
10. Remarks: The implementation shall provide overloads for all floating-point types as the type of parameter `value`.

```
to_chars_result to_chars(char* first, char* last, float value, chars_format fmt);  
to_chars_result to_chars(char* first, char* last, double value, chars_format fmt);  
to_chars_result to_chars(char* first, char* last, long double value, chars_format fmt);
```

```
to_chars_result to_chars(char* first, char* last, *see below* value, chars_format fmt);
```

10. *Requires*: `fmt` has the value of one of the enumerators of `chars_format`.
11. *Effects*: `value` is converted to a string in the style of `printf` in the "C" locale.
12. *Throws*: Nothing.
13. Remarks: The implementation shall provide overloads for all floating-point types as the type of parameter `value`.

```
to_chars_result to_chars(char* first, char* last, float value,  
                           chars_format fmt, int precision);  
to_chars_result to_chars(char* first, char* last, double value,  
                           chars_format fmt, int precision);  
to_chars_result to_chars(char* first, char* last, long double value,  
                           chars_format fmt, int precision);
```

```
to_chars_result to_chars(char* first, char* last, *see below* value,  
chars_format fmt, int precision);
```

13. *Requires:* `fmt` has the value of one of the enumerators of `chars_format`.
14. *Effects:* value is converted to a string in the style of `printf` in the `"C"` locale with the given precision.
15. *Throws:* Nothing.
16. *Remarks:* The implementation shall provide overloads for all floating-point types as the type of parameter `value`.

Modify Primitive numeric input conversions [**charconv.from.chars**]:

[...]

```
from_chars_result from_chars(const char* first, const char* last, float  
                             chars_format fmt = chars_format::general)  
from_chars_result from_chars(const char* first, const char* last, double  
                             chars_format fmt = chars_format::general)  
from_chars_result from_chars(const char* first, const char* last, long  
                             chars_format fmt = chars_format::general)
```

```
from_chars_result from_chars(const char* first, const char* last, *see  
                             chars_format fmt = chars_format::general)
```

6. *Requires*: `fmt` has the value of one of the enumerators of `chars_format`.

7. *Effects*: The pattern is the expected form of the subject sequence in the "C" locale, as described for `strtod`, except that

- (7.1) the sign '+' may only appear in the exponent part;
- (7.2) if `fmt` has `chars_format::scientific` set but not `chars_format::fixed`, the otherwise optional exponent part shall appear;
- (7.3) if `fmt` has `chars_format::fixed` set but not `chars_format::scientific`, the optional exponent part shall not appear; and
- (7.4) if `fmt` is `chars_format::hex`, the prefix "0x" or "0X" is assumed. [*Example*: The string 0x123 is parsed to have the value 0 with remaining characters x123. — *end example*]

In any case, the resulting value is one of at most two floating-point values closest to the value of the string matching the pattern.

8. *Throws*: Nothing.

9. *Remarks*: The implementation shall provide overloads for all floating-point types as the type of parameter value.

Note: other conversion to string functions (from `[strings]`) are not rewritten to support extended floating-point types.

Modify Complex numbers `[complex.numbers]` paragraph 2:

2. The effect of instantiating the template `complex` for any type ~~other than `float`, `double`, or `long double`~~ that is not a floating-point type is unspecified. The specializations ~~specializations `complex<float>`, `complex<double>`, and `complex<long double>`~~ of `complex` for floating-point types are literal types.

Modify Header `<complex>` synopsis [`complex.syn`]:

[...]

```
// [complex.special], specializations  
template<> class complex<float>;  
template<> class complex<double>;  
template<> class complex<long double>;
```

Modify Class template `complex` [`complex`]:

```

namespace std {
    template<class T> class complex {
    public:
        using value_type = T;

        constexpr complex(const T& re = T(), const T& im = T());

```

```

constexpr complex(const complex&);
template<class X> constexpr complex(const complex<X>&);

```

```

    constexpr complex(const complex&) = default;
    template<class X> constexpr explicit(*see below) complex(const comp

```

```

    constexpr T real() const;
    constexpr void real(T);
    constexpr T imag() const;
    constexpr void imag(T);

```

```

    constexpr complex& operator= (const T&);
    constexpr complex& operator+=(const T&);
    constexpr complex& operator-=(const T&);
    constexpr complex& operator*=(const T&);
    constexpr complex& operator/=(const T&);

```

```

    constexpr complex& operator=(const complex&);
    template<class X> constexpr complex& operator= (const complex<X>&);
    template<class X> constexpr complex& operator+=(const complex<X>&);
    template<class X> constexpr complex& operator-=(const complex<X>&);
    template<class X> constexpr complex& operator*=(const complex<X>&);
    template<class X> constexpr complex& operator/=(const complex<X>&);

```

```

    };
}

```

Remove Specializations [**complex.special**]:

```

namespace std {
template<> class complex<float> {
public:
using value_type = float;

constexpr complex(float re = 0.0f, float im = 0.0f);
constexpr complex(const complex<float>&) = default;
constexpr explicit complex(const complex<double>&);
constexpr explicit complex(const complex<long double>&);

constexpr float real() const;
constexpr void real(float);
constexpr float imag() const;
constexpr void imag(float);

constexpr complex& operator= (float);
constexpr complex& operator+=(float);
constexpr complex& operator=(float);
constexpr complex& operator*=(float);
constexpr complex& operator/=(float);

constexpr complex& operator=(const complex&);
template<class X> constexpr complex& operator= (const complex<X>&);
template<class X> constexpr complex& operator+=(const complex<X>&);
template<class X> constexpr complex& operator=(const complex<X>&);
template<class X> constexpr complex& operator*=(const complex<X>&);
template<class X> constexpr complex& operator/=(const complex<X>&);
};

template<> class complex<double> {
public:
using value_type = double;

constexpr complex(double re = 0.0, double im = 0.0);
constexpr complex(const complex<float>&);
constexpr complex(const complex<double>&) = default;
constexpr explicit complex(const complex<long double>&);

constexpr double real() const;
constexpr void real(double);
constexpr double imag() const;
constexpr void imag(double);

```

```

constexpr complex& operator= (double);
constexpr complex& operator+=(double);
constexpr complex& operator=(double);
constexpr complex& operator*=(double);
constexpr complex& operator/=(double);

constexpr complex& operator=(const complex&);
template<class X> constexpr complex& operator= (const complex<X>&);
template<class X> constexpr complex& operator+=(const complex<X>&);
template<class X> constexpr complex& operator=(const complex<X>&);
template<class X> constexpr complex& operator*=(const complex<X>&);
template<class X> constexpr complex& operator/=(const complex<X>&);
};

template<> class complex<long double> {
public:
using value_type = long double;

constexpr complex(long double re = 0.0L, long double im = 0.0L);
constexpr complex(const complex<float>&);
constexpr complex(const complex<double>&);
constexpr complex(const complex<long double>&) = default;

constexpr long double real() const;
constexpr void real(long double);
constexpr long double imag() const;
constexpr void imag(long double);

constexpr complex& operator= (long double);
constexpr complex& operator+=(long double);
constexpr complex& operator=(long double);
constexpr complex& operator*=(long double);
constexpr complex& operator/=(long double);

constexpr complex& operator=(const complex&);
template<class X> constexpr complex& operator= (const complex<X>&);
template<class X> constexpr complex& operator+=(const complex<X>&);
template<class X> constexpr complex& operator=(const complex<X>&);
template<class X> constexpr complex& operator*=(const complex<X>&);
template<class X> constexpr complex& operator/=(const complex<X>&);
};
}

```

■

Modify Member functions [**complex.members**] by inserting the following after paragraph 2:

```
template<class X> constexpr explicit(*see below*) complex(const complex<
```

3. *Effects*: Constructs an object of class `complex`.

4. *Ensures*: `real() == other.real() && imag() == other.imag()`.

5. *Remarks*: The expression inside `explicit` evaluates to `false` if and only if the range of finite values of `T` is a superset of the range of finite values of `X`.

Modify Additional overloads [**cmplx.over**] paragraph 2 and 3:

2. The additional overloads shall be sufficient to ensure:

- ~~(2.1) If the argument has type `long double`, then it is effectively cast to `complex<long double>`.~~
- ~~(2.2) Otherwise, if the argument has type `double` or an integer type, then it is effectively cast to `complex<double>`.~~
- ~~(2.3) Otherwise, if the argument has type `float`, then it is effectively cast to `complex<float>`.~~
- (2.1) If the argument has a floating-point type `T`, then it is effectively cast to `complex<T>`.
- (2.2) Otherwise, if the argument has an integer type, then it is effectively cast to `complex<double>`.

3. Function template `pow` shall have additional overloads sufficient to ensure, for a call with at least one argument of type `complex<T>`:

- ~~(3.1) If either argument has type `complex<long double>` or type `long double`, then both arguments are effectively cast to `complex<long double>`.~~
- ~~(3.2) Otherwise, if either argument has type `complex<double>`, `double`, or an integer type, then both arguments are effectively cast to `complex<double>`.~~
- ~~(3.3) Otherwise, if either argument has type `complex<float>` or `float`, then both arguments are effectively cast to `complex<float>`.~~
- (3.1) If the type of one of the arguments is `complex<T1>` and the type of the other is `complex<T2>`, then both arguments are effectively cast to `complex<TR>`, where `TR` is `T1` if `T1` has a higher floating-point conversion rank than `T2`, otherwise `T2`.
- (3.2) Otherwise, if the type of one of the arguments is `complex<T1>` and the type of the other is a floating-point type `T2`, then both arguments are effectively cast to `complex<TR>`, where `TR` is `T1` if `T1` has a higher floating-point conversion rank than `T2`, otherwise `T2`.
- (3.3) Otherwise, both arguments are effectively cast to `complex<T>`.

Modify Header `<cmath>` synopsis [`cmath.syn`] paragraph 2 and add paragraph 3:

2. For each set of overloaded functions within `<cmath>`, with the exception of `abs`, there shall be additional overloads sufficient to ensure:

1. ~~If any argument of arithmetic type corresponding to a double parameter has type long double, then all arguments of arithmetic type corresponding to double parameters are effectively cast to long double.~~
 2. ~~Otherwise, if any argument of arithmetic type corresponding to a double parameter has type double or an integer type, then all arguments of arithmetic type corresponding to double parameters are effectively cast to double.~~
 3. If all arguments of arithmetic types corresponding to `double` parameters have floating-point types, then all arguments of arithmetic type corresponding to `double` parameters have type that is the type among the argument types with the highest floating-point conversion rank. If that type is an extended floating-point type, then the return type is also that type.
 4. Otherwise, if any argument of arithmetic type corresponding to a `double` parameter has a floating-point type, then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to that of parameters of floating-point type that is the type with the highest floating-point conversion rank among those of argument types that are floating-point.
 5. Otherwise, all arguments of arithmetic type corresponding to `double` parameters have type `float`.
3. There shall be additional overloads of `abs` for each extended floating-point type `T`. Those overloads shall have the signature `T abs(T j)`, and return the absolute value of `j`.

Note: LWG question: should the signatures be somehow added to the synopsis itself?

Note: We have tried to capture what the current specification says, without having to add three identical items into this wording to cover, respectively, EFPTs bigger than `long double`, EFPTs between `double` and `float`, and EFPTs smaller than `float`. We don't have anything against reverting to that, but wanted to try this more generic way of describing the behavior.

Note: We are pretty sure this new paragraph 3 is not the way to spell it, so we will welcome any suggestions.

§ References

§ Informative References

[P0192]

Michał Dominiak; et al. [`short float` and fixed-size floating point types](https://wg21.link/P0192). URL: <https://wg21.link/P0192>

[P1468]

Michał Dominiak; Boris Fomitchev; Sergei Nikolaev. [Fixed-layout floating-point type aliases](https://wg21.link/P1468). URL: <https://wg21.link/P1468>